MULTITHREADING

1.) To Print Current Executing Thread

In [1]:
```
import threading
print('Current Executing Thread:',threading.current_thread().getName())
```

```
Current Executing Thread: MainThread
```

2.) Creating a Thread without using any class.

In [1]:
```
from threading import *
def display(): # function is the task that runs in a new thread
    for i in range(10):
        print('Child Thread')
# Creation of Thread Object to Execute Display
t=Thread(target=display) # creates a thread and sets its job to run display()
t.start() # runs display() in parallel with the main thread
for i in range(10):
   print('Main Thread')
```

```
Child ThreadMain Thread
Main Thread
Main Thread
Main Thread
Main Thread
Main Thread
Main Thread
Main Thread
Main Thread
Main Thread

Child Thread
Child Thread
Child Thread
Child Thread
Child Thread
Child Thread
Child Thread
Child Thread
Child Thread
```

In [6]:
```
from threading import *
def display():
    for i in range(10):
        print('Child Thread')
# Creation of Thread Object to Execute Display
t=Thread(target=display)
t.start()
for i in range(10):
     print('Main Thread')
```

```
Child ThreadMain Thread
Child Thread
Child Thread
Main Thread
Main Thread

Main Thread
Main Thread
Main Thread
Main Thread
Main Thread
Main Thread
Main Thread
Child Thread
Child Thread
Child Thread
Child Thread
Child Thread
Child Thread
Child Thread
```

3.) To Print Main and Child Thread Name

In [16]:
```
from threading import*
def display():
    print('Child Thread Name:',current_thread().getName())
    print('')
t=Thread(target=display)

t.start()
print('Main Thread Name:',current_thread().getName())
```

```
Child Thread Name: Thread-14

Main Thread Name: MainThread
```

Creating a Thread without extending Thread class.

In [18]:
```
from threading import *
class MyThread(Thread):
    def run(self):
        for i in range(10):
```

```
            print('Child Thread')
t=MyThread()
t.start()     # triggers the run() method in a separate thread.
for i in range(10):     # Both loops execute concurrently
    print('Main Thread')
# The output of Child Thread and Main Thread will be
# interleaved, depending on the OS thread scheduler.
```

```
Child ThreadMain Thread
Main Thread
Main Thread
Main Thread
Main Thread
Main Thread
Main Thread
Main Thread
Main Thread
Main Thread

Child Thread
Child Thread
Child Thread
Child Thread
Child Thread
Child Thread
Child Thread
Child Thread
Child Thread
```

4.) Creating a Thread by extending Thread class

In [2]:
```
# example of extending the Thread class
from time import sleep
from threading import Thread

# custom thread class
class CustomThread(Thread):
    # override the run function
    def run(self):
        # block for a moment
        sleep(1)
        # display a message
```

```
        print('This is coming from another thread')
# create the thread
thread = CustomThread()
# start the thread
thread.start()
# wait for the thread to finish
print('Waiting for the thread to finish')
thread.join()
```

```
Waiting for the thread to finish
This is coming from another thread
```

5.) Print Numbers and Letters Using Threads

In [1]:
```
import threading
import time
# Function to print numbers
def print_numbers():
    for i in range(1, 6):
        print(f"Number: {i}")
        time.sleep(1)
# Function to print letters
def print_letters():
    for letter in ['A', 'B', 'C', 'D', 'E']:
        print(f"Letter: {letter}")
        time.sleep(1)
# Create threads
t1 = threading.Thread(target=print_numbers)
t2 = threading.Thread(target=print_letters)
# Start threads
t1.start()
t2.start()
# Wait for threads to finish
t1.join()
t2.join()
print("Both threads completed.")
```

```
Number: 1
Letter: A
Number: 2
Letter: B
Number: 3
Letter: C
Number: 4
Letter: D
Number: 5
Letter: E
Both threads completed.
```

Multithreading Priority Queue

In [1]:
```
import queue
import threading
import time
thread_exit_Flag = 0
class sample_Thread (threading.Thread):
   def __init__(self, threadID, name, q):
      threading.Thread.__init__(self)
      self.threadID = threadID
      self.name = name
      self.q = q
   def run(self):
      print ("initializing " + self.name)
      process_data(self.name, self.q)
      print ("Exiting " + self.name)
# helper function to process data
def process_data(threadName, q):
   while not thread_exit_Flag:
      queueLock.acquire()
      if not workQueue.empty():
         data = q.get()
         queueLock.release()
         print ("% s processing % s" % (threadName, data))
      else:
         queueLock.release()
         time.sleep(1)
thread_list = ["Thread-1", "Thread-2", "Thread-3"]
```

```
name_list = ["A", "B", "C", "D", "E"]
queueLock = threading.Lock()
workQueue = queue.Queue(10)
threads = []
threadID = 1
# Create new threads
for thread_name in thread_list:
   thread = sample_Thread(threadID, thread_name, workQueue)
   thread.start()
   threads.append(thread)
   threadID += 1
# Fill the queue
queueLock.acquire()
for items in name_list:
   workQueue.put(items)
queueLock.release()
# Wait for the queue to empty
while not workQueue.empty():
   pass
# Notify threads it's time to exit
thread_exit_Flag = 1
# Wait for all threads to complete
for t in threads:
   t.join()
print ("Exit Main Thread")
```

```
initializing Thread-1
initializing Thread-2
initializing Thread-3
Thread-2 processing A
Thread-1 processing B
Thread-2 processing C
Thread-3 processing D
Thread-1 processing E
Exiting Thread-1Exiting Thread-3
Exiting Thread-2

Exit Main Thread
```

EXAMPLE-SCENARIO: "A food delivery application receives orders from multiple customers at the same time. "Each order needs to be processed independently. Write a simple Python program using "multithreading to process three customer orders simultaneously"

In [1]:
```python
import threading
import time

def process_order(customer):
    print(customer, "order received")
    time.sleep(2)   # Simulate order processing time
    print(customer, "order completed")


# Create threads for different customers
t1 = threading.Thread(target=process_order, args=("Arun",))
t2 = threading.Thread(target=process_order, args=("Priya",))
t3 = threading.Thread(target=process_order, args=("Kumar",))

# Start all threads
t1.start()
t2.start()
t3.start()

# Wait for all threads to complete
t1.join()
t2.join()
t3.join()

print("All orders processed")
```

```
ArunPriya order received
 order received
Kumar order received
PriyaArun order completed
Kumar order completed
 order completed
All orders processed
```

" Because the orders are independent, they can be processed concurrently "instead of waiting for one order to finish before starting the next"